CSV Loading Using csv.DictReader and LangChain Core

1. Today's Master Formula

CSV File

↓

csv.DictReader

↓

One Data Row

↓

One LangChain Document

Short form:

CSV → csv.DictReader → Document Per Data Row

2. What Is Our Requirement?

Suppose we have a CSV file:

students.csv

with this data:

student_id name course fee

S101 Ravi Python 5000

S102 Priya Java 6000

S103 Rahul Data Science 12000

We have:

3 Students

↓

3 Data Records

Therefore, we want:

3 LangChain Documents

Concept:

Row 1

↓

Document 1

Row 2

↓

Document 2

Row 3

↓

Document 3

3. Important CSV Document Structure

For Ravi, we may create a Document like this.

page_content

student_id: S101

name: Ravi

course: Python

fee: 5000

metadata


```
{
   "source": "students.csv",
   "row": 0,
   "source_type": "csv"
}
```


So:

page_content

↓

Actual Student Information

and:

metadata

↓

Information About

Where the Record Came From

4. Content vs Metadata

This distinction is very important.

Document

│

├── page_content

│ ↓

│ Student Data

│

└── metadata

↓

Source Information

Example:

page_content

---

student_id: S101

name: Ravi

course: Python

fee: 5000

Metadata:


```
{
   "source": "students.csv",
   "row": 0,
   "source_type": "csv"
}
```


5. Understanding CSV Field Selection

Earlier, while using the old CSVLoader, we studied parameters such as:

metadata_columns

content_columns

source_column

These were parameters of the old integration.

They are **not parameters of LangChain's Document class**.

But the underlying requirements are still useful.

We may still decide:

Which fields become page_content?

Which fields become metadata?

Which field becomes the record identifier?

We can implement all these decisions ourselves using:

csv.DictReader

+

Document

6. Modern CSV Loading Approach

Instead of depending on the older CSV loader integration:

CSV

↓

Old CSVLoader

↓

Document

we can directly use:

CSV

↓

Python csv.DictReader

↓

LangChain Core Document

This gives us full control over the data transformation.

7. What Is csv.DictReader?

csv.DictReader reads each CSV row as a Python dictionary.

Suppose our CSV contains:

student_id,name,course,fee

S101,Ravi,Python,5000

Then:

row

will conceptually contain:

```
{
   "student_id": "S101",
   "name": "Ravi",
   "course": "Python",
   "fee": "5000"
```

}

So:

CSV Header

↓

Dictionary Keys

and:

CSV Row Values

↓

Dictionary Values

8. DictReader Master Formula

CSV Row

↓

csv.DictReader

↓

Dictionary

Example:

S101,Ravi,Python,5000

becomes:

{

```
    "student_id": "S101",
    "name": "Ravi",
    "course": "Python",
    "fee": "5000"
}
```

9. Program 1 — Creating Documents From CSV Records

Objective

Read a CSV file using csv.DictReader and create one LangChain Document for each data row.

```
import csv


from pathlib import Path


from langchain_core.documents import Document



# ========================================
# CREATE SAMPLE CSV
# ========================================
```

```
file_path = (
    Path(__file__).resolve().parent
    / "students.csv"
)


file_path.write_text(
    "student_id,name,course,fee\n"
    "S101,Ravi,Python,5000\n"
    "S102,Priya,Java,6000\n"
    "S103,Rahul,Data Science,12000\n",
    encoding="utf-8"
)


# ========================================
# CREATE DOCUMENTS FROM CSV
# ========================================
```

```
documents = []


with file_path.open(
    "r",
    encoding="utf-8",
    newline=""
) as file:

    reader = csv.DictReader(
        file
    )


    for row_index, row in enumerate(
        reader
    ):

        content = "\n".join(
            f"{field}: {value}"
```

```
    for field, value in row.items()
)



documents.append(
    Document(
        page_content=content,

        metadata={
            "source":
                str(file_path),

            "source_type":
                "csv",

            "row":
                row_index
        }
    )
)
```

```
# ========================================
# DISPLAY RESULTS
# ========================================

print(
    "TOTAL DOCUMENTS:",
    len(documents)
)


for document in documents:

    print(
        "\nROW:",
        document.metadata["row"]
    )
```

```
    print(
        document.page_content
    )
```

10. Program 1 Flow

students.csv

↓

Open File

↓

csv.DictReader

↓

Dictionary Row

↓

Convert Row to Text

↓

Create Document

↓

Append to documents

↓

List of Documents

11. Creating the Sample CSV

We create:

```
file_path = (
    Path(__file__).resolve().parent
    / "students.csv"
)
```

Then:

```
file_path.write_text(
    "student_id,name,course,fee\n"
    "S101,Ravi,Python,5000\n"
    "S102,Priya,Java,6000\n"
    "S103,Rahul,Data Science,12000\n",
    encoding="utf-8"
)
```

creates the CSV file.

Concept:

Python Program

↓

write_text()

↓

students.csv

12. Opening the CSV File

We use:

```
with file_path.open(
    "r",
    encoding="utf-8",
    newline=""
) as file:
```

Important arguments:

"r"

↓

Read Mode

encoding="utf-8"

↓

Text Encoding

newline=""

↓

Do Not Modify

Original Line Endings

13. What Does newline="" Mean?

newline=""

means:

Python should not modify the original line-ending characters.
Let the CSV reader handle them.

Simple memory:

newline=""

↓

Let csv Module

Handle CSV Newlines

14. Creating DictReader

```
reader = csv.DictReader(
    file
)
```

This reads the CSV using the first row as field names.

For:

student_id,name,course,fee

the dictionary keys become:

student_id

name

course

fee

15. Understanding row

Inside:

```
for row_index, row in enumerate(
    reader
):
```

row is a dictionary.

For Ravi:

row

conceptually becomes:

```
{
    "student_id": "S101",
    "name": "Ravi",
    "course": "Python",
    "fee": "5000"
}
```

16. Understanding row.items()

row.items()

returns field-value pairs.

Conceptually:

student_id → S101

name → Ravi

course → Python

fee → 5000

So:

for field, value in row.items()

gives each:

Field Name

+

Field Value

17. Understanding This Expression

f"{field}: {value}"

For example:

field = "name"

value = "Ravi"

produces:

name: Ravi

Similarly:

course: Python

and:

fee: 5000

18. Understanding "\n".join()

We use:

```
content = "\n".join(
    f"{field}: {value}"
    for field, value in row.items()
)
```

This joins every field-value pair using a newline.

Result:

student_id: S101

name: Ravi

course: Python

fee: 5000

19. "\n".join() vs " ".join()

Using "\n".join()

student_id: S101

name: Ravi

course: Python

fee: 5000

Each field appears on a separate line.

Using " ".join()

student_id: S101 name: Ravi course: Python fee: 5000

Everything appears on one line.

20. Easy Memory

"\n".join()

↓

New Line Between Items

" ".join()

↓

One Space Between Items

21. Understanding enumerate(reader)

We use:

```
for row_index, row in enumerate(
    reader
):
```

This gives:

Row Index + Row Dictionary

Example:

0 → Ravi

1 → Priya

2 → Rahul

So:

row_index

↓

Zero-Based Data Row Number

22. Creating the Document

For every CSV row:

```
Document(
   page_content=content,

   metadata={
      "source": str(file_path),
      "source_type": "csv",
      "row": row_index
   }
)
```

is created.

Concept:

CSV Row

↓

content

↓

page_content

and:

File + Row Information

↓

metadata

23. Why Use documents.append()?

We begin with:

documents = []

For each row:

documents.append(

Document(...)

)

adds one Document to the list.

Concept:

Start

documents = []

After Ravi:

[

Document 1

]

After Priya:

[

Document 1,

Document 2

]

After Rahul:

[

Document 1,

Document 2,

Document 3

]

24. Expected Document Count

Our CSV contains:

3 Data Rows

Therefore:

len(documents)

returns:

3

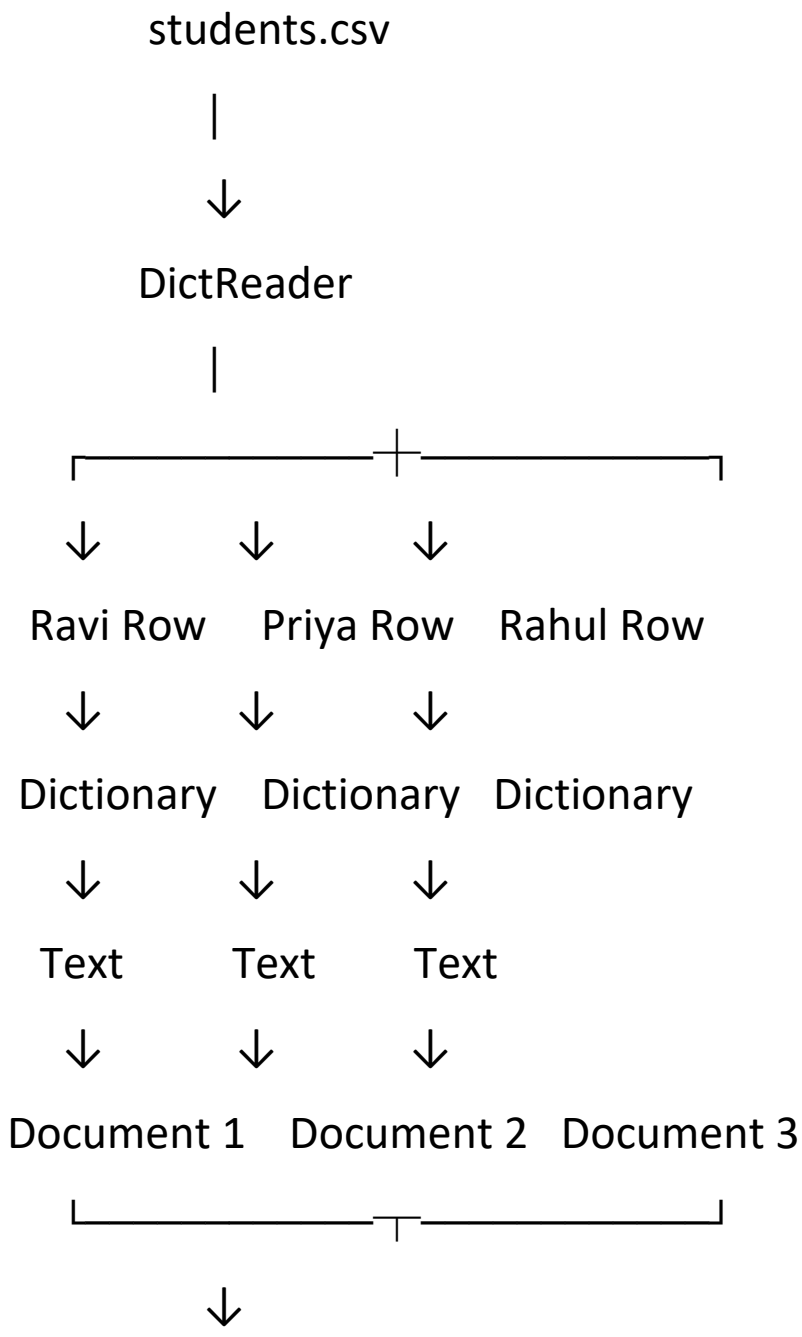
Master rule:

One Data Row

↓

One Document

25. Program 1 Final Architecture



List of Documents

26. Program 2 — Custom CSV Loader With Selected Content and Metadata

Objective

Use the maintained BaseLoader interface and control which CSV fields become:

page_content

and which become:

metadata

27. Requirement

Suppose CSV contains:

student_id

name

course

fee

But we want only:

name

fee

inside page_content.

And we want:

student_id

course

inside metadata.

So:

name + fee

↓

page_content

and:

student_id + course

↓

metadata

28. Program 2 — Complete Code

```
import csv

from pathlib import Path

from langchain_core.documents import Document
```

```
from langchain_core.document_loaders import (
    BaseLoader
)



# ========================================
# CUSTOM CSV LOADER
# ========================================

class StudentCSVLoader(
    BaseLoader
):

    def __init__(
        self,
        file_path
    ):

        self.file_path = Path(
            file_path
```

```
    )


def lazy_load(self):

    with self.file_path.open(
        "r",
        encoding="utf-8",
        newline=""
    ) as file:


        reader = csv.DictReader(
            file
        )


        for row_index, row in enumerate(
            reader
        ):
```

```
content = (
    f"name: {row['name']}\n"
    f"fee: {row['fee']}"
)


yield Document(
    page_content=content,

    metadata={
        "source":
            str(self.file_path),

        "source_type":
            "csv",

        "row":
            row_index,
```

```
                "student_id":
                    row["student_id"],

                "course":
                    row["course"]
            }
        )


# ========================================
# CREATE SAMPLE FILE
# ========================================

file_path = (
    Path(__file__).resolve().parent
    / "student_fees.csv"
)


file_path.write_text(
```

```
    "student_id,name,course,fee\n"
    "S101,Ravi,Python,5000\n"
    "S102,Priya,Java,6000\n",
    encoding="utf-8"
)



# ========================================
# LOAD STUDENT DOCUMENTS
# ========================================

loader = StudentCSVLoader(
    file_path
)



documents = loader.load()



# ========================================
```

```
# DISPLAY RESULTS
# ========================================

print(
    "TOTAL DOCUMENTS:",
    len(documents)
)


for document in documents:

    print(
        "\nCONTENT:"
    )


    print(
        document.page_content
    )
```

```
print(
    "STUDENT ID:",
    document.metadata[
        "student_id"
    ]
)


print(
    "COURSE:",
    document.metadata[
        "course"
    ]
)
```

29. Understanding StudentCSVLoader

We create:

```
class StudentCSVLoader(
   BaseLoader
):
```

This means:

StudentCSVLoader

↓

Inherits From

↓

BaseLoader

So our loader follows the LangChain loader interface.

30. What Does the Constructor Do?

```
def __init__(
   self,
   file_path
):
```

receives the CSV path.

Then:

```
self.file_path = Path(
    file_path
)
```

converts it into a Path object.

31. What Does lazy_load() Do?

```
def lazy_load(self):
```

contains the actual CSV loading logic.

The flow is:

Open CSV

↓

Create DictReader

↓

Read One Row

↓

Create Content

↓

Create Metadata

↓

yield Document

32. Selecting Fields for page_content

We create:

```
content = (
    f"name: {row['name']}\n"
    f"fee: {row['fee']}"
)
```

So for Ravi:

name: Ravi

fee: 5000

Only these fields appear in:

page_content

33. Selecting Fields for Metadata

We use:

```
metadata={
    "source":
        str(self.file_path),

    "source_type":
        "csv",
```

```
    "row":
        row_index,

    "student_id":
        row["student_id"],

    "course":
        row["course"]
}
```

So metadata contains:

source

source_type

row

student_id

course

Example:

```
{
    "source": "student_fees.csv",
    "source_type": "csv",
```

```
  "row": 0,
  "student_id": "S101",
  "course": "Python"
}
```

34. Program 2 Document Structure

For Ravi:

```
Document
  |
  ├── page_content
  |     ↓
  |   name: Ravi
  |   fee: 5000
  |
  └── metadata
        ↓
      source
      source_type
      row
      student_id
```

course

35. Old CSVLoader Concept vs Custom Loader

Earlier we had:

content_columns

Now we manually decide what goes into:

page_content

Earlier:

metadata_columns

Now we manually decide what goes inside:

metadata={...}

Earlier:

source_column

Now we can manually decide what should be used as:

metadata["source"]

36. Important Concept

The old parameters are gone from this custom implementation:

metadata_columns

content_columns

source_column

But the **design ideas remain**.

We still ask:

What data should the LLM/retriever read?

↓

page_content

and:

What data should describe or identify

the record?

↓

metadata

37. Why Use a Custom BaseLoader?

A custom loader gives us:

Control

Standardization

Reusability

LangChain Compatibility

Instead of repeating CSV conversion code everywhere, we create:

StudentCSVLoader

and reuse it.

38. lazy_load() vs load()

Our custom class implements:

lazy_load()

It yields one Document at a time.

Concept:

CSV Row 1

↓

yield Document 1

CSV Row 2

↓

yield Document 2

Then:

documents = loader.load()

uses the inherited loader interface to return the Documents as a list.

Easy memory:

lazy_load()

↓

YIELD

load()

↓

LIST

39. Program 2 Flow

student_fees.csv

↓

StudentCSVLoader

↓

BaseLoader

↓

lazy_load()

↓

DictReader

↓

Read Row

↓

Select Content Fields

↓

name + fee

↓

page_content

At the same time:

student_id

course

source

row

↓

metadata

Then:

yield Document

↓

load()

↓

List of Documents

40. Program 1 vs Program 2

Program 1	**Program 2**
Direct Python approach	Custom loader approach
Uses csv.DictReader	Uses csv.DictReader
Creates Documents directly	Yields Documents through BaseLoader
All fields become content	Selected fields become content
Basic metadata	Custom metadata
Good for understanding logic	Better reusable loader structure

41. Important Python Concepts Used

csv.DictReader

CSV Row

↓

Dictionary

enumerate()

Row

+

Index

row.items()

Dictionary

↓

Field-Value Pairs

join()

Many Strings

↓

One String

yield

Produce One Document

At a Time

BaseLoader

Standard LangChain

Loader Interface

42. Common Mistake — Thinking Header Becomes a Document

Suppose:

student_id,name,course,fee

S101,Ravi,Python,5000

S102,Priya,Java,6000

The first line is the header.

It provides field names.

The data rows are:

Ravi

Priya

So:

2 Data Rows

↓

2 Documents

not three.

43. Common Mistake — Thinking DictReader Returns a List

Inside iteration:

row

is a dictionary-like row.

Concept:

DictReader

↓

One Row at a Time

↓

Dictionary

44. Common Mistake — Confusing row With Row Number

In:

```
for row_index, row in enumerate(
   reader
):
```

these are different.

row_index

↓

0, 1, 2...

whereas:

row

↓

CSV Data Dictionary

45. Common Mistake — Thinking newline="" Removes Newlines

No.

newline=""

does not mean:

Remove Newlines

It means:

Do Not Transform Line Endings

↓

Let csv Module Handle Them

46. Common Mistake — Thinking "\n".join() and " ".join() Are Same

They differ only in the separator used.

"\n".join(...)

produces:

field1: value1

field2: value2

Whereas:

" ".join(...)

produces:

field1: value1 field2: value2

47. Common Mistake — Thinking Every Field Must Be in page_content

No.

We decide which fields belong in:

page_content

and which belong in:

metadata

Example:

name + fee

↓

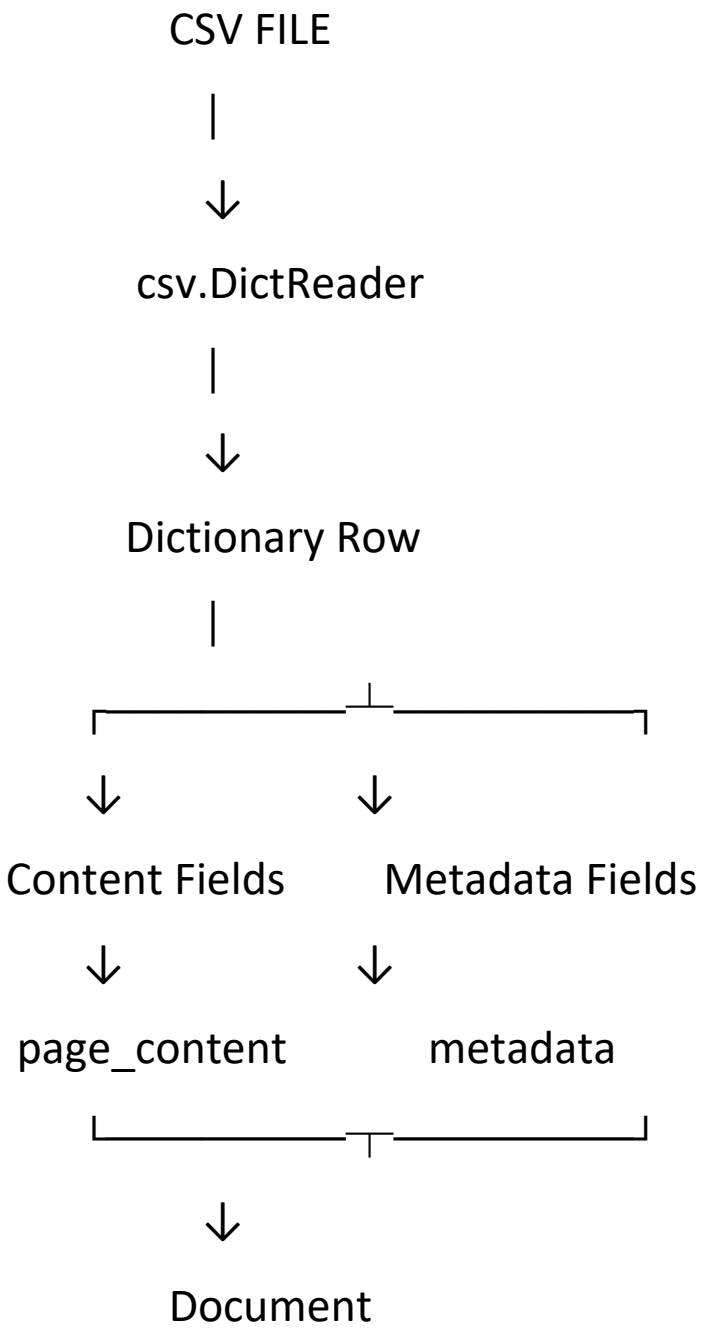
page_content

student_id + course

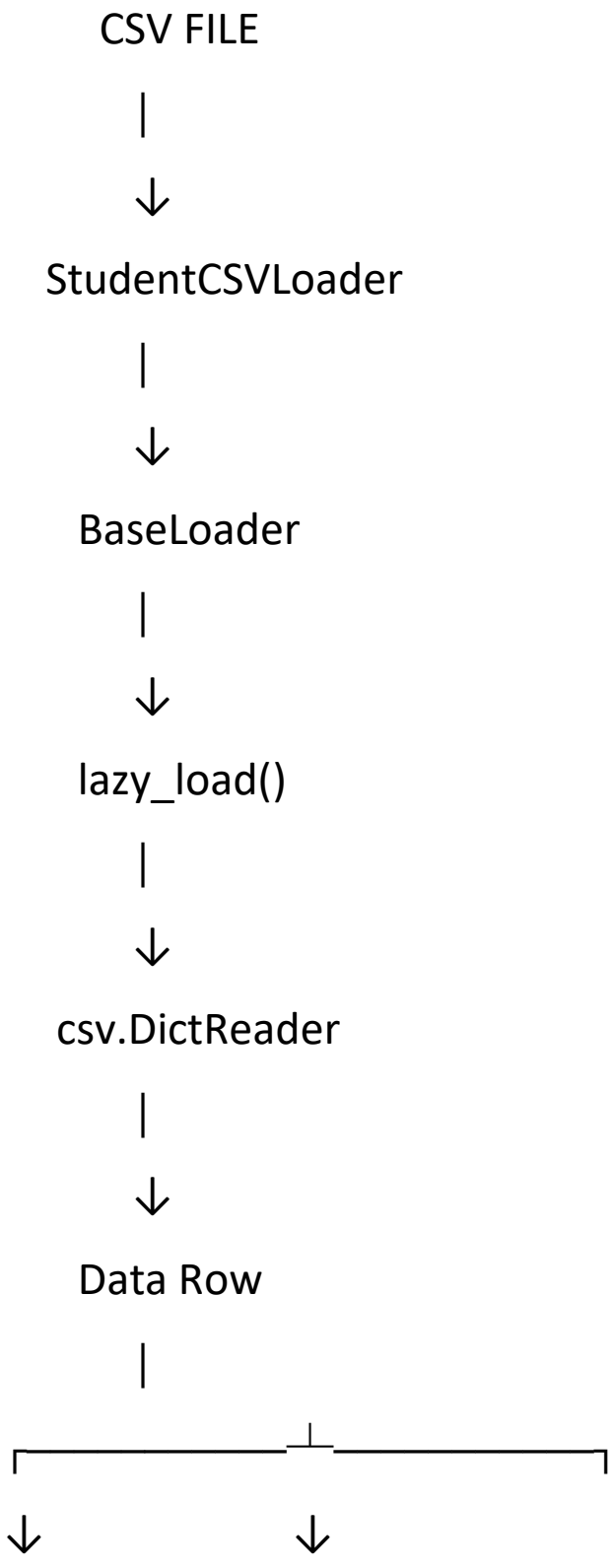
↓

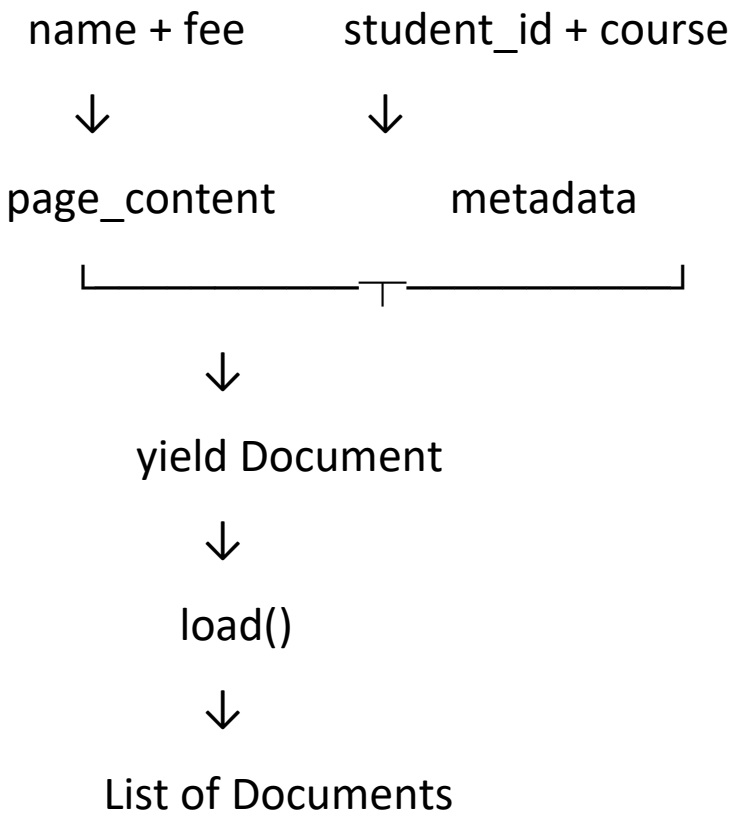
metadata

48. Modern CSV Loading Architecture



49. Custom Loader Architecture





50. Important Notes

1. csv.DictReader reads each CSV row as a dictionary.
2. One CSV data row can become one LangChain Document.
3. CSV headers become dictionary field names.
4. page_content stores the textual representation of the record.
5. Metadata stores source and identification information.
6. row can be stored as a zero-based record index.
7. newline="" lets the CSV module handle line endings.

8. "\n".join() places each field-value pair on a separate line.
9. " ".join() places field-value pairs on one line.
10. We can manually choose which CSV fields go into content and metadata.
11. BaseLoader lets us turn this logic into a reusable LangChain-compatible loader.
12. A custom lazy_load() can yield one Document for each CSV row.

51. Today's Master Comparison

DIRECT APPROACH

==========

CSV

↓

DictReader

↓

Loop

↓

Document

↓

documents.append()

CUSTOM LOADER APPROACH

==========

CSV

↓

StudentCSVLoader

↓

BaseLoader

↓

lazy_load()

↓

DictReader

↓

yield Document

↓

load()

↓

List[Document]

52. Today's Master Formula

CSV

↓

csv.DictReader

↓

Dictionary Row

↓

Document

For reusable LangChain loading:

CSV

↓

Custom BaseLoader

↓

csv.DictReader

↓

lazy_load()

↓

Document Per Data Row

↓

List of Documents

53. One-Line Conclusion

Use Python's csv.DictReader to convert each CSV data row into structured field-value data, then create LangChain Document objects with selected information in page_content and useful identification details in metadata; for reusable applications, place this logic inside a custom BaseLoader.